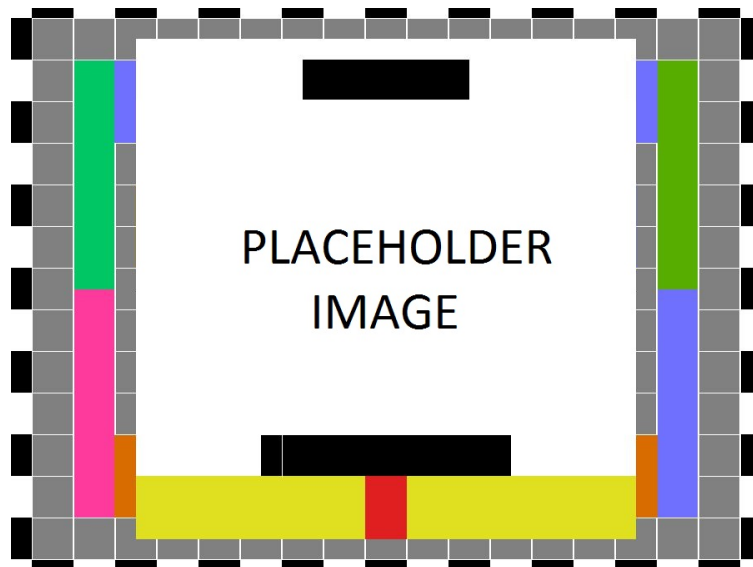


**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
THE UNIVERSITY OF TEXAS AT ARLINGTON**

**ARCHITECTURAL DESIGN SPECIFICATION
CSE 4316: SENIOR DESIGN I
FALL 2026**



**TEAM NAME
PRODUCT NAME**

ALAN TURING
GRACE HOPPER
JOHN VON NEUMANN
ADA LOVELACE
CHARLES BABBAGE

REVISION HISTORY

Revision	Date	Author(s)	Description
0.1	10.01.2026	GH	document creation
0.2	10.05.2026	AT, GH	complete draft
0.3	10.12.2026	AT, GH	release candidate 1
1.0	10.20.2026	AT, GH, CB	official release
1.1	10.31.2026	AL	added design review requests

CONTENTS

1	Introduction	5
1.1	Purpose & Scope	5
1.2	Applicable Documents	5
2	Stakeholders & Architectural Concerns	6
2.1	Stakeholders	6
2.2	Architectural Concerns	6
3	Architecture Viewpoints	7
3.1	Architectural Style & Conventions	7
4	System Overview & Context View	8
4.1	Layer X Description	8
4.2	Layer Y Description	8
4.3	Layer Z Description	8
5	Subsystem Definitions & Data Flow	9
6	X Layer Subsystems	10
6.1	Subsystem 1	10
6.2	Subsystem 2	11
6.3	Subsystem 3	11
7	Y Layer Subsystems	12
7.1	Subsystem 1	12
7.2	Subsystem 2	13
7.3	Subsystem 3	13
8	Z Layer Subsystems	14
8.1	Subsystem 1	14
8.2	Subsystem 2	15
8.3	Subsystem 3	15
9	Architecture Decisions & Rationale	16
9.1	AD-01: [Decision Title]	16
9.2	AD-02: [Decision Title]	16
10	Requirements Traceability	17

LIST OF FIGURES

1	Architectural layer diagram	8
2	A simple data flow diagram	9
3	Example subsystem description diagram	10
4	Example subsystem description diagram	12
5	Example subsystem description diagram	14

LIST OF TABLES

2	Stakeholder / concern matrix	6
3	Architecture viewpoints used in this document	7
4	Subsystem interfaces	11
5	Subsystem interfaces	13
6	Subsystem interfaces	15
7	Requirement-to-architecture traceability (excerpt)	17

1 INTRODUCTION

This Architectural Design Specification (ADS) describes the software and system architecture of Product Name. It is organized as an *architecture description* in the sense of ISO/IEC/IEEE 42010:2022 [1] and draws its design viewpoints from IEEE Std 1016-2009 [2]. The architecture is presented as a set of complementary *views*, each governed by a *viewpoint* that frames the concerns of specific stakeholders.

Your introduction should describe the product concept in sufficient detail that the architectural design is easy to follow. The introduction may reuse the product overview from the System Requirements Specification (SRS) for this purpose. At a minimum, state the product concept, scope, and key requirements, and reference the SRS and Project Charter so the reader can locate the driving requirements. Avoid detailed implementation choices (specific components, languages) here — those belong to the Detailed Design Specification.

1.1 PURPOSE & SCOPE

State the purpose of this document and the scope of the architecture it describes. Identify what is inside the architecture's boundary and what is treated as an external actor or system.

1.2 APPLICABLE DOCUMENTS

Reference the SRS (source of requirements), the Project Charter (source of constraints and objectives), and any external standards or interface control documents the architecture must conform to.

2 STAKEHOLDERS & ARCHITECTURAL CONCERNS

ISO/IEC/IEEE 42010:2022 [1] requires that an architecture description identify the system's *stakeholders* and the *concerns* they hold about the architecture. Concerns are the interests — such as functionality, performance, safety, security, cost, maintainability, or regulatory compliance — that the architecture must address. Each concern is later framed by one or more architecture viewpoints (Section 3) and answered by the corresponding views (Sections 4–8).

2.1 STAKEHOLDERS

Identify each stakeholder or stakeholder role that has an interest in the architecture. Typical roles include the customer/sponsor, end users, the development team, maintainers, the course instructor, and any external system owners.

2.2 ARCHITECTURAL CONCERNS

List the concerns held by the stakeholders above. Focus on concerns that shape architectural decisions (not detailed requirements). The matrix below maps stakeholders to their concerns; use it to confirm every significant concern has an owner and is addressed by at least one view.

Table 2: Stakeholder / concern matrix

Stakeholder	Primary concerns
Customer / sponsor	functionality delivered, cost, schedule, fitness for purpose
End users	usability, responsiveness, reliability
Development team	feasibility, modularity, testability
Maintainers	modifiability, observability, documentation
(add roles)	(add concerns)

3 ARCHITECTURE VIEWPOINTS

An *architecture viewpoint* (ISO/IEC/IEEE 42010:2022 [1]) establishes the conventions for constructing and interpreting one kind of *view*: which stakeholders and concerns it frames, and the model kinds (notations) it uses. This section declares the viewpoints used in the remainder of the document. Each subsequent section is a view that conforms to one of these viewpoints. The viewpoints below combine the 42010 framework with the design viewpoints of IEEE Std 1016-2009 [2]; adapt the set to your project.

Table 3: Architecture viewpoints used in this document

Viewpoint	Frames (concerns)	Model kind	Realized in
Context	system boundary, external actors/systems	context diagram	Section 4
Composition / Data Flow	decomposition into layers/subsystems and the data exchanged	block & data-flow diagram	Section 5
Logical / Functional	responsibilities and interfaces of each subsystem	block diagram, interface tables	Sections 6–8
Deployment	mapping of software to hardware/nodes	deployment diagram	Section 7 or 8 (as needed)

3.1 ARCHITECTURAL STYLE & CONVENTIONS

Describe the overall architectural strategy for the system: the top-level decomposition into *layers*. An architectural layer is the top-level logical view, or abstraction, of the design. Layers are composed of related elements of similar capability, are highly independent of other layers, and have clearly defined interfaces and interactions. Each layer is identified individually and is unique in its function and purpose within the system.

State the naming conventions your team will use for layers, subsystems, modules, interfaces, and data flows, and the interface identifier scheme (e.g. #01, #02) referenced by the subsystem interface tables in Sections 6–8. Consistent conventions here make the views, the Detailed Design Specification, and the traceability matrix easy to cross-reference.

4 SYSTEM OVERVIEW & CONTEXT VIEW

This section presents the **context view** and the top-level structure of the system. The context view (conforming to the Context viewpoint declared in Section 3) fixes the system boundary and shows the external actors and systems that interact with Product Name. The layer descriptions then give the overall structure of the software system — the strategy for how it will be built.

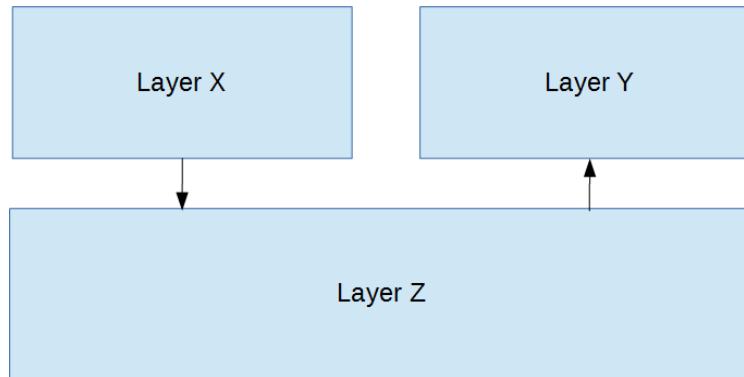


Figure 1: Architectural layer diagram

An architectural “layer” is the top-level logical view, or abstraction, of your design. Layers should be composed of related elements of similar capability, should be highly independent of other layers, and should have clearly defined interfaces and interactions with other layers. Each layer is identified individually and is unique in its function and purpose within the system. Provide the high-level block diagram of the layers above, and describe the function of each layer below.

4.1 LAYER X DESCRIPTION

Describe this layer in detail: its features, functions, and the services it provides, and its critical interfaces and interactions with adjacent layers. Note any architecturally significant properties (e.g. this layer is the only one permitted to touch external hardware, or persistent storage).

4.2 LAYER Y DESCRIPTION

Describe this layer in detail, following the same pattern as Layer X.

4.3 LAYER Z DESCRIPTION

Describe this layer in detail, following the same pattern as Layer X. Ensure the set of layers is complete (every major responsibility belongs to exactly one layer) and that inter-layer interfaces are consistent with the data-flow view in Section 5.

5 SUBSYSTEM DEFINITIONS & DATA FLOW

This section breaks down your layer abstraction to another level of detail. Here you graphically represent the logical subsystems that compose each layer and show the interactions/interfaces between those subsystems. A subsystem can be thought of as a programming unit that implements one of the major functions of the layer. It, therefore, has data elements that serve as source/sinks for other subsystems. The logical data elements that flow between subsystems need to be explicitly defined at this point, beginning with a data flow-like diagram based on the block diagram.

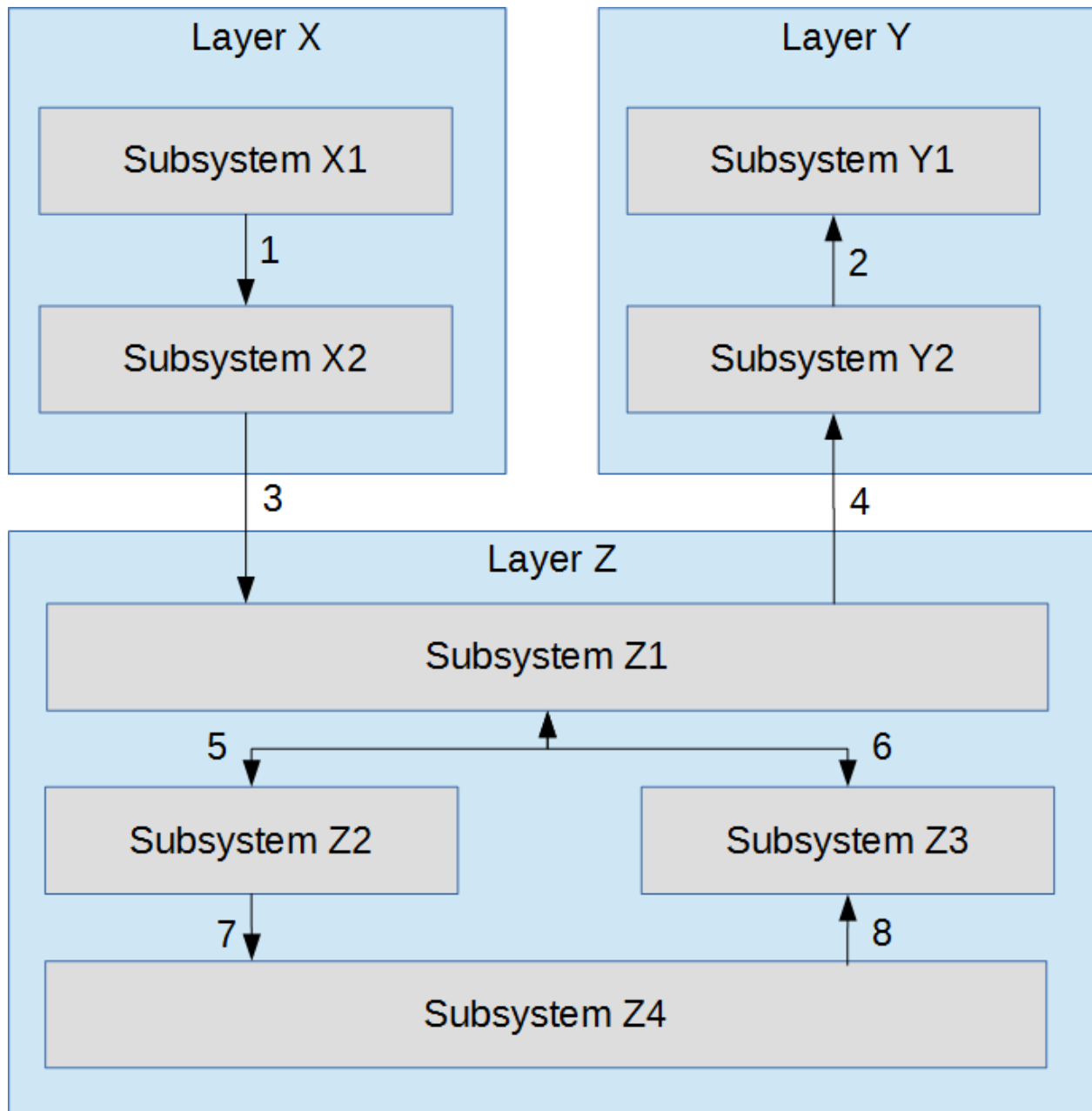


Figure 2: A simple data flow diagram

6 X LAYER SUBSYSTEMS

This section is the **logical/functional view** of the X layer (conforming to the Logical/Functional viewpoint declared in Section 3). The layer is decomposed into its subsystems, and the responsibilities, interfaces, and data flows of each subsystem are defined. A subsystem is a programming unit that implements one of the major functions of the layer and serves as a source/sink of data for other subsystems. Describe each subsystem in a separate subsection using the same structure. Include any special considerations or trade-offs for the approach chosen.

6.1 SUBSYSTEM 1

A general description of this subsystem. For most subsystems, an extract of the architectural block diagram with data flows is useful — showing this subsystem and those it communicates with.

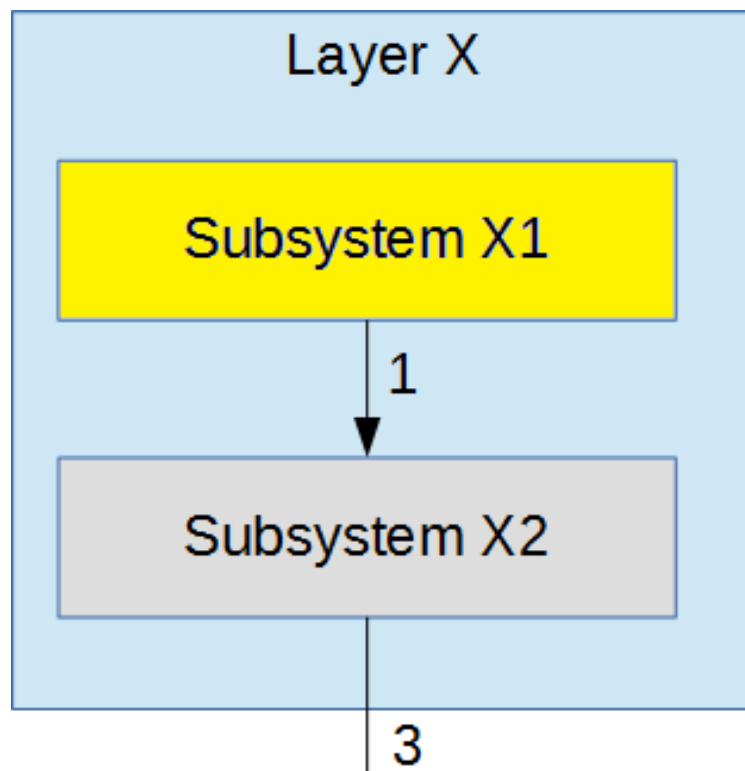


Figure 3: Example subsystem description diagram

6.1.1 ASSUMPTIONS

Any assumptions made in defining the subsystem. Pay particular attention to assumptions concerning interfaces and interactions with other layers.

6.1.2 RESPONSIBILITIES

Expand each responsibility/feature/function/service of this subsystem, as identified in the architectural overview, into more detailed responsibilities. Clearly describe what the subsystem does (and does not do).

6.1.3 SUBSYSTEM INTERFACES

Define each input and output for the subsystem. Create one row per labelled interface that connects to this subsystem, describing the incoming and outgoing data elements. Interface identifiers use the

scheme declared in Section 3 and are the anchors for requirements traceability.

Table 4: Subsystem interfaces

ID	Description	Inputs	Outputs
#01	Description of the interface/bus	input 1 input 2	output 1
#02	Description of the interface/bus	N/A	output 1

6.1.4 DESIGN RATIONALE

Briefly justify this subsystem's boundary and approach: why these responsibilities are grouped together, and any significant alternatives considered. Record architecturally significant choices as decisions in Section 9 and reference them here (e.g. "per AD-02").

6.2 SUBSYSTEM 2

Repeat for each subsystem.

6.3 SUBSYSTEM 3

Repeat for each subsystem.

7 Y LAYER SUBSYSTEMS

This section is the **logical/functional view** of the Y layer (conforming to the Logical/Functional viewpoint declared in Section 3). The layer is decomposed into its subsystems, and the responsibilities, interfaces, and data flows of each subsystem are defined. A subsystem is a programming unit that implements one of the major functions of the layer and serves as a source/sink of data for other subsystems. Describe each subsystem in a separate subsection using the same structure. Include any special considerations or trade-offs for the approach chosen.

7.1 SUBSYSTEM 1

A general description of this subsystem. For most subsystems, an extract of the architectural block diagram with data flows is useful — showing this subsystem and those it communicates with.

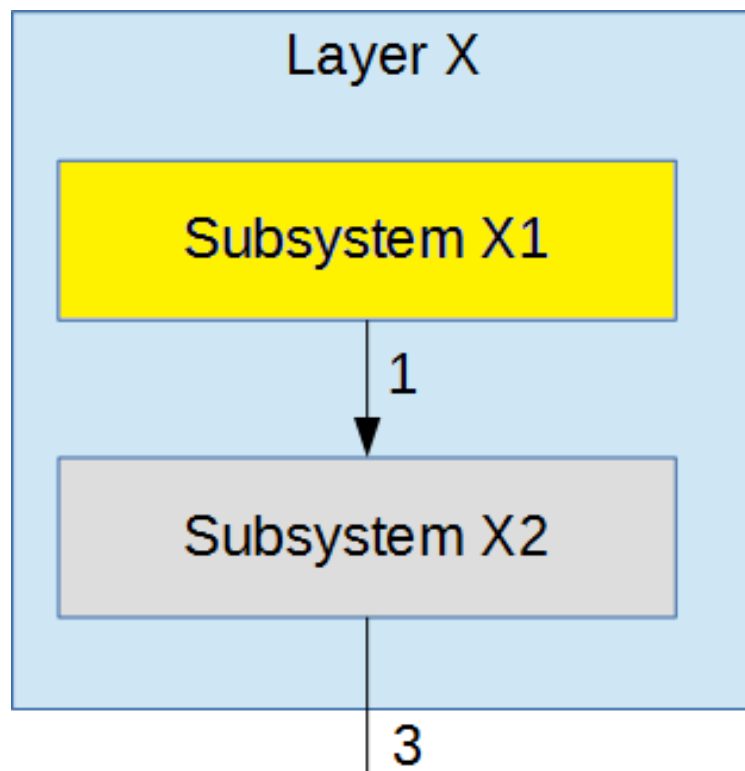


Figure 4: Example subsystem description diagram

7.1.1 ASSUMPTIONS

Any assumptions made in defining the subsystem. Pay particular attention to assumptions concerning interfaces and interactions with other layers.

7.1.2 RESPONSIBILITIES

Expand each responsibility/feature/function/service of this subsystem, as identified in the architectural overview, into more detailed responsibilities. Clearly describe what the subsystem does (and does not do).

7.1.3 SUBSYSTEM INTERFACES

Define each input and output for the subsystem. Create one row per labelled interface that connects to this subsystem, describing the incoming and outgoing data elements. Interface identifiers use the

scheme declared in Section 3 and are the anchors for requirements traceability.

Table 5: Subsystem interfaces

ID	Description	Inputs	Outputs
#01	Description of the interface/bus	input 1 input 2	output 1
#02	Description of the interface/bus	N/A	output 1

7.1.4 DESIGN RATIONALE

Briefly justify this subsystem's boundary and approach: why these responsibilities are grouped together, and any significant alternatives considered. Record architecturally significant choices as decisions in Section 9 and reference them here (e.g. "per AD-02").

7.2 SUBSYSTEM 2

Repeat for each subsystem.

7.3 SUBSYSTEM 3

Repeat for each subsystem.

8 Z LAYER SUBSYSTEMS

This section is the **logical/functional view** of the Z layer (conforming to the Logical/Functional viewpoint declared in Section 3). The layer is decomposed into its subsystems, and the responsibilities, interfaces, and data flows of each subsystem are defined. A subsystem is a programming unit that implements one of the major functions of the layer and serves as a source/sink of data for other subsystems. Describe each subsystem in a separate subsection using the same structure. Include any special considerations or trade-offs for the approach chosen.

8.1 SUBSYSTEM 1

A general description of this subsystem. For most subsystems, an extract of the architectural block diagram with data flows is useful — showing this subsystem and those it communicates with.

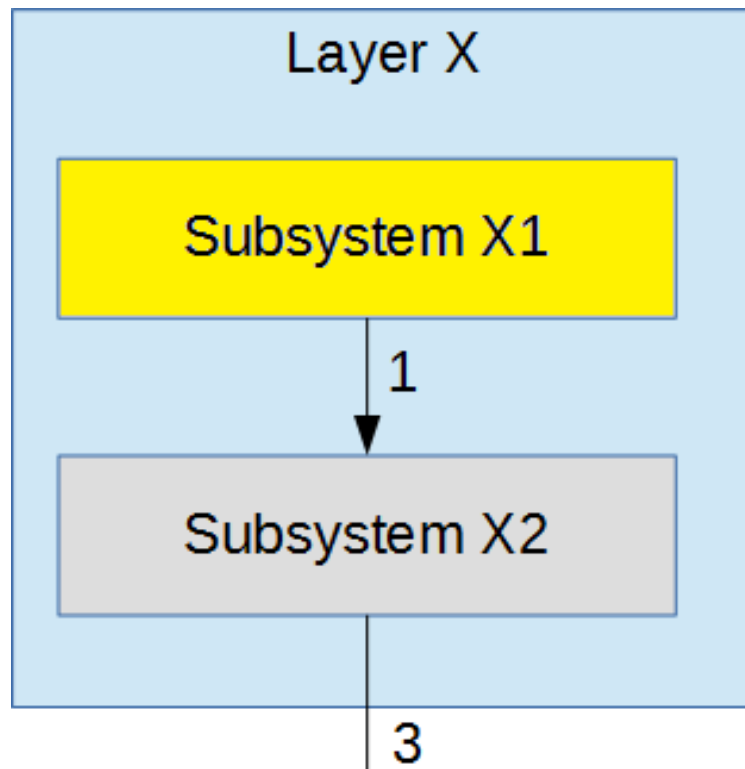


Figure 5: Example subsystem description diagram

8.1.1 ASSUMPTIONS

Any assumptions made in defining the subsystem. Pay particular attention to assumptions concerning interfaces and interactions with other layers.

8.1.2 RESPONSIBILITIES

Expand each responsibility/feature/function/service of this subsystem, as identified in the architectural overview, into more detailed responsibilities. Clearly describe what the subsystem does (and does not do).

8.1.3 SUBSYSTEM INTERFACES

Define each input and output for the subsystem. Create one row per labelled interface that connects to this subsystem, describing the incoming and outgoing data elements. Interface identifiers use the

scheme declared in Section 3 and are the anchors for requirements traceability.

Table 6: Subsystem interfaces

ID	Description	Inputs	Outputs
#01	Description of the interface/bus	input 1 input 2	output 1
#02	Description of the interface/bus	N/A	output 1

8.1.4 DESIGN RATIONALE

Briefly justify this subsystem's boundary and approach: why these responsibilities are grouped together, and any significant alternatives considered. Record architecturally significant choices as decisions in Section 9 and reference them here (e.g. "per AD-02").

8.2 SUBSYSTEM 2

Repeat for each subsystem.

8.3 SUBSYSTEM 3

Repeat for each subsystem.

9 ARCHITECTURE DECISIONS & RATIONALE

ISO/IEC/IEEE 42010:2022 [1] calls for an architecture description to record the *architecture decisions* and their *rationale* — the reasoning, alternatives considered, and trade-offs behind the architecture. Capturing decisions here makes the architecture reviewable and gives future maintainers the “why” behind the structure. Record each architecturally significant decision using the lightweight format below (an architecture decision record).

9.1 AD-01: [DECISION TITLE]

- **Context:** The situation and forces (requirements, constraints, concerns) that made a decision necessary.
- **Decision:** The choice that was made, stated plainly.
- **Alternatives considered:** Other options evaluated and why they were not chosen.
- **Consequences:** The resulting trade-offs — positive and negative — and any concerns (from Section 2) that remain only partially addressed.
- **Affected concerns / requirements:** Stakeholder concerns and SRS requirement IDs influenced by this decision.

9.2 AD-02: [DECISION TITLE]

Repeat for each architecturally significant decision (for example: choice of layering, monolith vs. service decomposition, synchronous vs. message-based communication, on-device vs. cloud processing, primary datastore, safety-interlock strategy).

10 REQUIREMENTS TRACEABILITY

This section provides forward traceability from the requirements baseline (SRS) to the architectural elements that realize each requirement. It confirms that every significant requirement is addressed by at least one subsystem, and that every subsystem exists to satisfy identified requirements. Maintain this matrix as the architecture evolves; the Detailed Design Specification extends it down to individual components, and the Test Plan links requirements to test cases.

Table 7: Requirement-to-architecture traceability (excerpt)

Req ID	Requirement (abbreviated)	Realizing subsystem(s)
FR-01	abbreviated requirement text	X Layer / Subsystem 1
PE-01	abbreviated requirement text	Y Layer / Subsystem 2
SE-01	abbreviated requirement text	Z Layer / Subsystem 1

REFERENCES

- [1] ISO/IEC/IEEE, “ISO/IEC/IEEE 42010:2022 — Software, systems and enterprise — Architecture description,” International Organization for Standardization, Geneva, Switzerland, Tech. Rep., 2022.
- [2] IEEE, “IEEE Std 1016-2009 — IEEE Standard for Information Technology — Systems Design — Software Design Descriptions,” Institute of Electrical and Electronics Engineers, New York, NY, USA, Tech. Rep., 2009.